

FoSyMa Cours 6

Coordination et Planification multiagent

Aurélie Beynier

FoSyMa, Master 1 ANDROIDE

17 février 2025

Plan

- 1 Caractéristiques des agents
- 2 Coordination
- 3 Organisation
- 4 Coordination par planification
- 5 Représentation du problème / des plans
 - STRIPS multi-agent
 - PDDL multi-agent
- 6 Allocation des buts

Caractéristiques des agents

Examinons plus en détails les caractéristiques principales des agents dans un SMA :

- Distribution
- Asynchronisme
- Ouverture
- Couplage
- Homogénéité
- Rationalité
- Coordination

Distribution

Distribution conceptuelle

- Un agent est une entité autonome ayant ses propres connaissances, préférences, comportements.
- Certaines données (et même la plupart) d'un agent sont privées.
- Chaque agent n'a qu'une observabilité partielle du système (environnement + autres agents). En cas d'observabilité totale, le système n'est plus distribué, ce n'est pas un SMA !
- Chaque agent prend ses propres décisions à partir de sa propre expertise et de son point de vue sur le système.

Distribution

Distribution logicielle

- Au niveau implémentation machine, chaque agent est un processus propre et a son propre cycle de vie
- Les agents s'exécutent en parallèle : 1 agent s'exécute sur 1 processus
- Le plus souvent 1 agent $\iff$ 1 objet
- Chaque agent a ses propres données.
- Plusieurs types d'agents peuvent être définis avec des comportements différents.

Distribution

En Jade

- En Jade, 1 agent s'exécute sur un Thread qui lui est propre
- Ne pas utiliser de variables partagées entre les agents, les agents s'échangent des données par envoi de messages.
- Les containers de Jade peuvent être distribués sur le réseau donc les agents peuvent s'exécuter sur des machines différentes. Le *MainContainer* est unique et représente le point de lancement de la plateforme.

Distribution

Distribution physique

- Les agents peuvent s'exécuter sur différentes machines
 - Communication entre les machines à prendre en compte (souvent coûteux)
 - Perte de réseaux et pannes machines à gérer
 - Possibilité de migration des agents (agents mobiles)
- Les agents s'exécutent au sein de différentes plateformes
 - Nécessité d'interfacer les plateformes
 - Par exemple : JADE et Jason

Asynchronisme

Chaque agent a sa propre vitesse d'exécution

- Les agents ne sont pas forcément au même point dans leur cycle d'exécution
- En décision séquentielle : les agents ne prennent pas tous une décision au même moment
- Impact important sur la communication : lorsqu'un agent reçoit un message, il ne le traite pas immédiatement
 - Problème de famine : on arrive pas à "attraper" la ressource
 - Interactions : saturation des boîtes aux lettres, manque de réactivité lors des interactions

Asynchronisme

Pistes envisageables :

- Possibilité de mettre les agents en attente afin de favoriser la synchronisation : pas de garantie que les agents soient toujours correctement synchronisés
- Mise en place d'un agent "horloge" permettant de synchroniser les autres agents :
 - Point critique du système
 - Attention au cas où les agents communiquent de manière asynchrone : l'heure envoyée par l'horloge n'est pas traitée instantanément
- Utiliser des horloges propres à chaque agent :
 - Horloge Logique de Lamport (cf. cours 10)
 - Chaque agent a un compteur d'évènements
 - Les agents communiquent les données et la valeur du compteur

Asynchronisme

Pistes envisageables (suite) :

- Développer des systèmes robustes à l'asynchronisme → cela reste la meilleure solution
 - Par exemple : à base d'accusés de réception et de timeout (cf. ContractNet)
 - Nécessite un effort de conception, encore plus important lorsque le système est ouvert !

Asynchronisme

En Jade

- Deux agents de la même classe, lancés en même temps peuvent être à des points différents de leur exécution
- Un agent ne traite pas le message immédiatement lors de sa réception
- Lorsque l'exécution d'un comportement est lancée, celui-ci s'exécute jusqu'à ce qu'il se termine : pas d'entrelacement de comportements au cours de leur exécution → éviter les "gros comportements fourre-tout"
- Recours à une horloge centrale difficile

Ouverture

Dans un système **ouvert**, le nombre d'agents et leur type n'est pas connu *a priori*. [Sab09]

- Des agents peuvent entrer et sortir du système à tout moment
- Nécessité de pouvoir identifier les agents et les services qu'ils offrent
- Les comportements des agents doivent tenir compte de la dynamique des agents avec lesquels il est possible d'interagir

Ouverture

En Jade

- Mécanismes de naissance et de mort (bien définir les méthodes `setup` et `takedown`)
- Agent Management System supervise l'enregistrement des agents (pages blanches)
- Directory Facilitator (architecture FIPA) permettant aux agents d'enregistrer les services qu'ils proposent (pages jaunes)
- Mobilité intra-plateforme et inter-plateforme : `void doMove(Location destination)`

Couplage

Des **agents faiblement couplés** sont des agents qui ont été conçus de manière indépendante (par exemple par différents concepteurs).

- Les agents n'ont donc pas de connaissance *a priori* sur le fonctionnement des autres agents.
- Si les agents sont homogènes, ils peuvent toutefois interagir (hétérogénéité $\neq$ faiblement couplés).
- Nécessité de définir des protocoles d'interaction

Homogénéité

Hétérogénéité syntaxique

Les agents ne partagent pas les mêmes protocoles d'interaction

- Si associé à un faible couplage → on ne peut rien faire !
- En pratique, ce cas se pose rarement.
- En JADE : agents homogènes syntaxiquement, utilisation de FIPA/ACL

Homogénéité

Hétérogénéité sémantique

Les agents ne partagent pas la même représentation conceptuelle du monde.

- Par exemple, pour exprimer le fait qu'un agent est mort, un agent utilise `mort(Agent_i)` et l'autre `décédé(Agent_i)`
- C'est pour éviter ce problème que l'on définit des **ontologies**

Ontologie

Une **ontologie** est la spécification des concepts, objets et relations d'un domaine donné.

- Un agent exprime ses connaissances en utilisant le vocabulaire lié à une ontologie.
- L'ontologie est un dénominateur commun entre les agents qui leur permet de se comprendre.

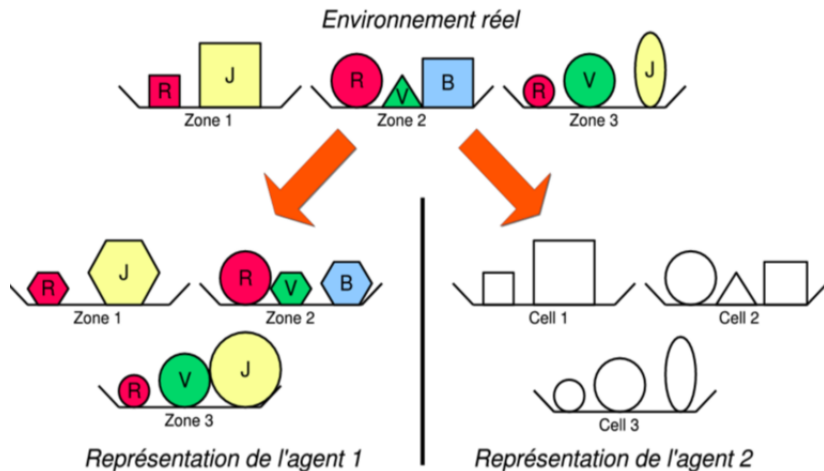
Homogénéité

Hétérogénéité sémantique et ontologies

Lorsque chaque agent utilise sa propre ontologie (par exemple dans des systèmes ouverts), il est possible d'utiliser des méthodes d'**alignement d'ontologies**

- Soit 2 agents a_1 et a_2 avec respectivement les ontologies A et B
- Aligner les ontologies A et B consiste "à trouver un ensemble d'associations entre un ensemble de concepts de l'ontologie A et un ensemble de concepts de l'ontologie B " [Sab09]
- Cet alignement doit pouvoir être calculable en temps réel et être réalisé automatiquement

Alignements d'ontologies



Ontologies

Langages d'ontologies

- **XML (et RDF)** : pas un langage d'ontologies à la base mais permet la définition rapide d'ontologies simples
- **OWL** (Web Ontology Language) : extension de RDF fournissant des outils permettant de raisonner sur les connaissances représentées (par exemple réaliser des comparaisons de propriétés et de classes)
- **KIF** : langage basé sur la logique du premier ordre

Ontologies

Contenu des messages en JADE

- Utilisées lors de l'envoi de messages
- Plusieurs possibilités :
 - Utiliser des chaînes de caractères : nécessité de parser toute la chaîne à chaque message
 - Utiliser la sérialisation Java : ne permet pas de communiquer avec d'autres plateformes non Java, le récepteur ne sait pas quel type d'objet il reçoit
 - XML : nécessité d'écrire des encodeurs/décodeurs XML
 - Ontologies de JADE : code Java, permet de vérifier la sémantique des messages et de réaliser la transformation en chaîne de caractères

Ontologies

Les ontologies en JADE

Pour définir une ontologie, on crée une instance de la classe `jade.content.onto.Ontology`

- Définir les prédicats (`PredicateSchema`), actions (`AgentActionSchema`) et concepts (`ConceptSchema`)
- Déclarer les composant des prédicats, actions et concepts
- Concepts : entités qui existent dans le monde (personne, adresse,...) (`Person :name John :age 33`)
- Prédicats : expriment le statut d'une partie du monde. Peut être vrai ou faux : (`Works-for (Person :name John) (Company :name Sorbonne)`)
- Actions : actions que l'agent peut être amené à faire (`Sell (Book :title 'The Lord of the rings') (Person :name John)`)

Ontologies

Chaque concept et prédicat doit ensuite être implémenté dans une classe spécifique :

```
public class Book implements Concept {...}
```

avec les accesseurs correctement définis. Par exemple :

```
public void setTitle(String s){...}
```

```
public String getTitle(){...}
```

Ontologies

Il faut ensuite sélectionner un langage de contenu et enregistrer le langage et l'ontologie auprès de l'agent:

```
public class MonAgent extends Agent{

    private Codec codec = new SLCodec();
    private Ontology ontology = MonOntology.getInstance();

    protected void setup(){

        getContentManager().registerLanguage(codec);
        getContentManager().registerOntology(ontology);
        ....
    }
}
```

Ontologies

Enfin, on peut définir des messages utilisant cette ontologie :

```
// Prepare the Query-IF message
ACLMessage msg = new ACLMessage(ACLMessage.QUERY_IF);
msg.addReceiver(sellerAID) // sellerAID is the AID of the Seller a
msg.setLanguage(codec.getName());
msg.setOntology(ontology.getName());

// Prepare the content
Book bo = new Book();
bo.setTitle("?JADE and Ontologies?);

try {
    // Let JADE convert from Java objects to string
    getContentManager().fillContent(msg, bo);
    send(msg);
}
```

Rationalité

Agent rationnel

Un agent rationnel est un agent qui agit “ au mieux ”, il prend à tout moment la meilleure décision.

Mathématiquement, cela peut se traduite par l'optimisation d'une fonction caractérisant les préférences ou les objectifs de l'agent → mesure de performance (agent cognitif)

Définition de Russel et Norvig [RN03]

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationalité

- Critère de performance défini par le concepteur
- Doit tenir compte des actions immédiates et également des effets à long terme
- Optimisation étant données les connaissances courantes de l'agent
- Point de vue subjectif

Rationalité

Modèles orientés but

- Classification des comportements en souhaités/ non souhaités
- Les comportements souhaités sont ceux permettant d'atteindre les buts
- Pas de préférences entre les comportements souhaités

Modèles à base d'utilité

- États : décrit les situations rencontrées en cours d'exécution
- Fonction d'utilité : définit des préférences sur les états, associe à chaque état un nombre réel décrivant le degré de satisfaction de l'agent dans cet état
- Un agent rationnel est un agent qui maximise son utilité
- Sous incertitude, définition de l'utilité espérée [VNM44]

A réfléchir ...

Dans le cadre du projet FoSyMa, traduire ce que signifie les notions suivantes :

- Distribution
- Asynchronisme
- Ouverture
- Couplage
- Homogénéité
- Rationalité
- Coordination

Coordination

Contexte

- Un ensemble de n **agents cognitifs interagissant dans un même environnement**
- Chaque agent a une **observabilité partielle** du système
- Les agents peuvent avoir des capacités différentes
- Les agents peuvent être coopératifs ou égoïstes
- Pas d'hypothèse *a priori* sur les capacités de communication

Problématique

Chaque agent doit être en mesure de **décider de manière autonome** (décision distribuée) quelles actions effectuer **en tenant compte de leur impact (intentionnel ou non) sur les possibilités futures d'actions.**

Coordination

Définition [Mal88]

“Ensemble des **activités supplémentaires** qu'il est nécessaire d'accomplir dans un environnement multi-agents et qu'un seul agent poursuivant les mêmes buts n'accomplirait pas.”

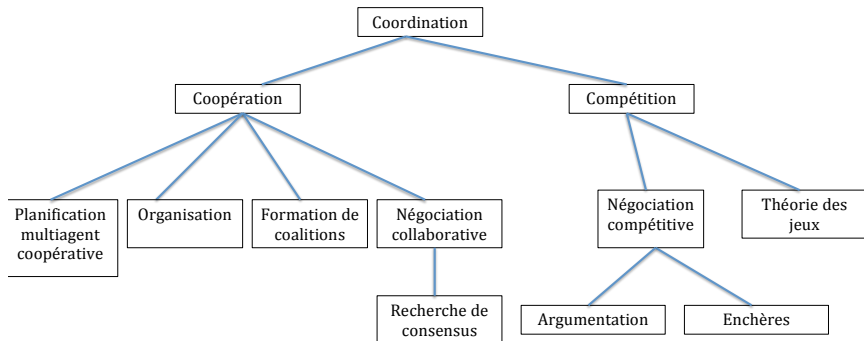
Définition 2 [Fer95]

“la coordination des actions, **dans le cadre de la coopération**, peut donc être définie comme l'articulation des actions individuelles accomplies par chacun des agents de manière à ce que l'ensemble aboutisse à un **tout cohérent et performant**. (...) l'action du groupe est améliorée soit par une **augmentation des performances**, soit par une **diminution des conflits**. La coordination des actions est donc l'une des principales méthodes pour assurer la coopération entre agents autonomes”

Coordination

- La coordination est utilisée dans le cas où les actions individuelles des agents peuvent interférer entre elles (interactions positives ou négatives)
 - Interactions positives : partage d'actions, réalisation conjointe d'actions, facilitation d'actions
 - Interactions négatives : incompatibilité de buts, accès à des ressources limitées
- Elle permet d'améliorer le comportement des agents en prenant en compte les interactions des comportements : résolution de blocages, amélioration des comportements, obtention de nouveaux comportements
- Par exemple : passage d'un couloir étroit, exploration partagée de l'espace, possibilité de soulever un objet lourd ou de détecter de nouveaux événements

Coordination



Qu'est-ce qu'une organisation

Définition Morin - [Fer95]

Une organisation peut être définie comme un agencement de relations entre composants ou individus qui produit une unité, ou système, dotée de qualités inconnues au niveau des composants ou individus. L'organisation lie de façon interrelationnelle des éléments ou événements ou individus divers dès lors deviennent les composants d'un tout. Elle assure solidarité et solidité relative, donc assure au système une certaine possibilité de durée en dépit de perturbations aléatoires.

Les organisations constituent à la fois le support et la manière dont se passent les interrelations, c'est-à-dire la façon dont sont réparties les tâches, les informations, les ressources et la coordination.

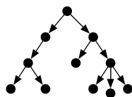
Structures organisationnelles

Structure organisationnelle

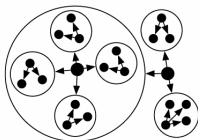
- La structure organisationnelle est ce qui caractérise, sur un plan abstrait, une organisation. Elle définit les rôles, les comportements attendus et les relations d'autorité entre les agents.
- L'organisation est une instantiation d'une structure.
- La structure organisationnelle est définie par un ensemble de classes d'agents caractérisés par les rôles qui leurs sont affectés et par un ensemble de relations entre ces rôles. Les rôles définissent les capacités, les objectifs, les droits et les obligations des différents partis.

Structures organisationnelles

- **Hiérarchiques** : sous forme d'arbre. Les nœuds de plus haut niveau ont une vue plus globale. Les agents n'interagissent qu'avec des entités directement connectées (niveau supérieur ou inférieur).

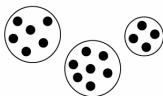


- **Holarchies** : organisation hiérarchiques entre holons. Chaque holon est une unité de base de l'organisation et fonctionne également comme un tout.



Structures organisationnelles

- **Coalitions** : chaque coalition est formée par un groupe d'agents interagissant ensemble. Il n'y a pas de coordination entre les différentes coalitions en dehors des buts qu'elles poursuivent. L'organisation au sein d'une coalition est généralement égalitaire mais on peut envisager une hiérarchie ou la présence d'un leader.



- **Egalitaires** : tous les agents ont le même “pouvoir”. Tous les agents peuvent demander à n'importe quel autre agent d'effectuer une tâche.
- ... [HL04]

Structures organisationnelles

Les structures organisationnelles peuvent être constituées de 2 manières :

- **prédéfinie** par le concepteur
- **émergente** : agents réactifs sans structure organisationnelle prédéfinie mais résultant des interactions entre les agents. La répartition des rôles et des tâches se fait par auto-organisation. Par exemple, les sociétés de fourmis.

Limitations des organisations

- Les organisations permettent de structurer les interactions, réduire et gérer l'incertitude.
- Toutefois, les organisations peuvent affecter l'efficacité, les coûts de communication, réduire la flexibilité et la réactivité.
→ Nécessité de penser minutieusement les structures d'organisation.

Coordination par planification

Difficultés du problème

- **Interactions :**
 - Dans un SMA, les actions d'un agent peuvent influencer les actions des autres agents (actions possibles et effets des actions, interactions positives ou négatives).
 - Un agent doit donc tenir compte de l'influence de ses actions sur les autres mais également de l'impact des actions des autres sur lui-même.
 - L'espace des trajectoires d'actions possibles peut croître de façon exponentielle → la plupart du temps impossible de toutes les énumérer.
- **Observabilité partielle :** un agent observe rarement les états et actions des autres agents ainsi que l'ensemble de l'environnement dans lequel il évolue.

Planification multi-agent

Exemples d'applications :

- Missions multi-robots : exploration, patrouille, assistance, ...



- Véhicules intelligents



- Jeux



Planification multi-agent

- Simulation de systèmes complexes : simulation urbaine, phénomènes économiques, mise en œuvre de politiques publiques, phénomènes biologiques, ...



- Smart cities



Planification multi-agent

- Aide à la décision dans des systèmes humains-logiciels :
patrouille de police, aiguilleurs du ciel, ...



- Planification de déplacements (par exemple systèmes GPS)
- Planification d'arguments à poser dans un débat



Planification multi-agent

Qu'est-ce qu'un plan ?

Séquence d'actions (partiellement) ordonnées dont l'exécution permet d'atteindre les buts souhaités ou de maximiser une mesure de performance.

Problématiques

- Les actions d'un agent ont des conséquences à plus ou moins long terme sur les autres agents (actions, objectifs).
- Il n'est pas efficace que les agents planifient indépendamment les uns des autres.
- **Coordination** requise entre les agents (qu'ils soient coopératifs ou compétitifs).

Planification multi-agent

- Planification **pour** plusieurs agents : les plans calculés seront exécutés en parallèle et de manière distribuée
- Planification **par** plusieurs agents : les agents collaborent afin de construire un plan ou un ensemble de plans.
 - Découpage du problème initiale en sous-problèmes
 - Résolution en parallèle des différentes parties du problème (calcul de plans)
 - Coordination entre les agents afin d'obtenir une solution globale cohérente et efficace
 - Évite le goulot d'étranglement et la fragilité liée au panne d'un point central.

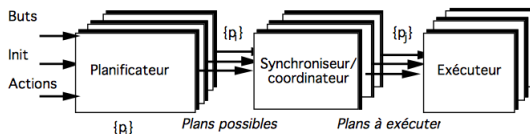
Différentes étapes de la planification multi-agent

- ➊ Allocation des buts aux agents
- ➋ Calcul de plans pour atteindre les buts
- ➌ Communication entre les agents et coordination des plans
- ➍ Exécution des plans

Ces phases ne sont pas toujours exécutées de manière séquentielle et peuvent s'intercaler les unes entre les autres.

Différentes étapes de la planification multi-agent

Une fois les buts et les tâches définis et alloués aux agents, le processus de planification peut être décomposé en 2 étapes distinctes [Fer95]



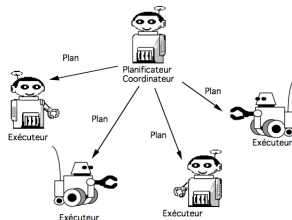
- Planification centralisée : 1 seul planificateur
- Planification distribuée : 1 planificateur par agent

Dans tous les cas, **les plans seront exécutés de manière distribuée.**

Principe de la planification centralisée

- Un agent coordinateur construit les plans individuels des agents en garantissant leur coordination
- Les plans sont ensuite distribués aux agents qui les exécutent individuellement
- Suppose que le planificateur possède une connaissance exacte du monde et soit capable de prévoir précisément son évolution.
- Les agents doivent également envoyer leurs connaissances privées au planificateur (préférences par exemple).

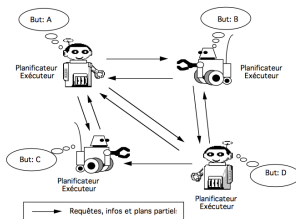
Figure extraite de [Fer95]



Principe de la planification distribuée

- Chaque agent calcule son propre plan en fonction de ses objectifs, de ses connaissances du monde et des informations individuelles qu'il possède sur les buts et les plans des autres agents.
- Pas d'agent central pour planifier ou coordonner les plans
- Les agents doivent communiquer afin de coordonner leurs plans individuels : que communiquer ? quand communiquer ? comment exploiter les informations reçues ?

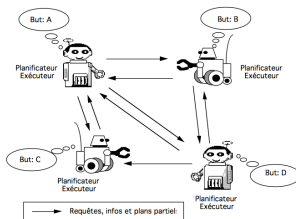
Figure extraite de [Fer95]



Principe de la planification distribuée

- Le processus de coordination des plans doit permettre d'aboutir à une solution stable et de bonne qualité (dans l'idéal l'optimal).
- Les agents peuvent conserver des informations privées.
- Le système est plus robuste aux défaillances des communications et des agents.
- Difficultés majeures : reconnaissance des situations d'interactions et résolution des conflits

Figure extraite de [Fer95]



Questions soulevées par la planification multi-agent

[dDOW99]

- Comment représenter le problème de planification ?
- Comment les tâches sont-elles allouées aux agents ?
- Comment les plans ou plans partiels sont-ils représentés ?
- Quelles sont les méthodes de base pour générer des plans ?
- Comment les agents communiquent-ils les uns avec les autres durant la planification ?
- Comment les agents coordonnent-ils leurs actions ?

Représentation du problème

Un problème de planification est défini par :

- Un ensemble fini d'**agents** $\mathcal{N} = \{1, 2, \dots, n\}$.
- Un ensemble fini d'**actions** pouvant être exécutées par ces agents $\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_n\}$ où $\mathcal{A}_i$ est l'ensemble des actions pouvant être exécutées par l'agent i .
 - Ces actions peuvent avoir des pré-conditions d'exécution et ont des effets sur le système (modification de l'environnement par exemple).
 - Le résultat d'une action jointe $a = \{a_1, a_2, \dots, a_n\}$ peut être différent des issues des actions individuelles : interactions (positives ou négatives) entre les actions des agents.
- Un ensemble de **buts** à atteindre
 - des états buts identifiés
 - une fonction objectif à maximiser (fonction de récompense)

Représentation du problème et des plans

Chaque agent doit donc pouvoir décider, à partir de ses connaissances individuelles, quelle(s) action(s) individuelle(s) réaliser.

Une grande variété de représentations de plans possibles, entre autres :

- Formalismes dérivés de STRIPS : pré-condition, action, post-condition
- Fonction *Etat* $\rightarrow$ *Action* ou *Connaissances* $\rightarrow$ *Action*
- Automate à états finis
- Réseaux de Pétri récursifs

Aparté sur STRIPS

- Langage “classique” de représentation des problèmes de planification basé sur la logique
- Les **états** sont représentés par une conjonction de littéraux propositionnels positifs et des littéraux de premier ordre :
 $\text{Clear} \wedge \text{Safe}$ ou $\text{At}(\text{Wumpus}, \text{House})$
- Les **buts** sont des états partiellement spécifiés représentant une conjonction de littéraux positifs. $\text{At}(\text{Tresor}, \text{House})$
- Les actions sont spécifiées par des pré-conditions devant être vérifiées pour exécuter l'action et des effets.

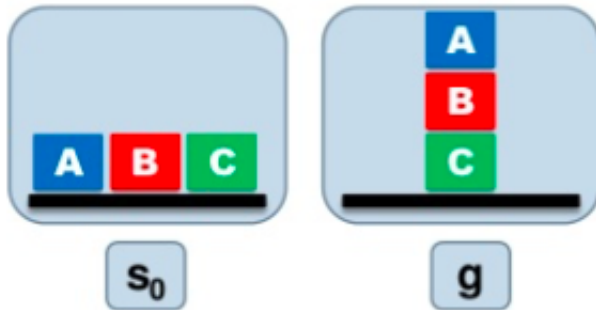
$\text{Move}(\text{Robot}, x, y) :$

pre : $\text{At}(\text{Robot}, x) \wedge \text{clear}(y)$

effect : $\text{At}(\text{Robot}, y) \wedge \text{clear}(x) \wedge \neg \text{clear}(y)$

Aparté sur STRIPS

Exemple du monde des blocs :



Aparté sur STRIPS

Exemple du monde des blocs - Représentation STRIPS :

Init($\text{On}(A, \text{Table}) \wedge \text{On}(B, \text{Table}) \wedge \text{On}(C, \text{Table}) \wedge \text{Block}(A) \wedge$
 $\text{Block}(B) \wedge \text{Block}(C) \wedge \text{Clear}(A) \wedge \text{Clear}(B) \wedge \text{Clear}(C)$)

Goal($\text{On}(A, B) \wedge \text{On}(B, C)$)

Action (Move(b, x, y),

Pre : $\text{On}(b, x) \wedge \text{Clear}(b) \wedge \text{Clear}(y) \wedge \text{Block}(b) \wedge (b \neq x)$
 $\wedge (b \neq y) \wedge (x \neq y)$

Effect : $\text{On}(b, y) \wedge \text{Clear}(x) \wedge \neg \text{On}(b, x) \wedge \neg \text{Clear}(y)$)

Action(MoveToTable(b, x))

Pre : $\text{On}(b, x) \wedge \text{Clear}(b) \wedge \text{Block}(b) \wedge (b \neq x)$

Effect : $\text{On}(b, \text{Table}) \wedge \text{Clear}(x) \wedge \neg \text{On}(b, x)$)

Représentation par MA-STRIPS [BD08]

- MA-STRIPS : Extension de STRIPS pour la représentation de problèmes de planification multi-agent.
- Un problème de planification en MA-STRIPS est représenté par :
 - Un ensemble fini d'agents $\mathcal{N} = \{1, 2, \dots, n\}$.
 - Un ensemble fini d'atomes ou de propositions $\mathcal{P}$ (permettant de décrire l'ensemble des états).
 - Un ensemble fini d'**actions** $\mathcal{A}_i$ pour chaque agent i . Chaque action individuelle α est définie par des pré-conditions, des effets d'ajout et des effets de suppression :
$$\alpha = \{pre(\alpha), add(\alpha), del(\alpha)\}.$$
 - $\mathcal{I} \subseteq \mathcal{P}$ définit l'état initial du système.
 - $\mathcal{G} \subseteq \mathcal{T}$ définit le but commun des agents.

Représentation par MA-STRIPS [BD08]

- Une solution au problème de planification est un ensemble ordonné d'actions dont l'exécution à partir de $\mathcal{I}$ mène à un état $\mathcal{S}_g$ tel que $\mathcal{G} \subseteq \mathcal{S}_g$.
- Afin de tenir compte de la distribution des agents, chaque atome $p \in \mathcal{P}$ est classé comme état **interne** ou **public**.
 - Un atome interne est uniquement utilisé et affecté par les actions d'un agent i .
 - Un atome public est utilisé et affecté par tous les agents.
- Les atomes accessibles pour un agent i sont donc $\mathcal{P}_i = \mathcal{P}_i^{int} \cup \mathcal{P}^{pub}$.
- Les actions d'un agent i sont alors partitionnées $\mathcal{A}_i = \mathcal{A}_i^{int} \cup \mathcal{A}_i^{pub}$ telles que les actions dans $\mathcal{A}_i^{int}$ ne contiennent que des atomes internes et les actions dans $\mathcal{A}_i^{pub}$ contiennent des atomes publics.

Représentation par MA-STRIPS [BD08]

- Un plan pour un agent i est un ensemble ordonné d'actions $a_{i1}, a_{i2} \cdots a_{iT}$ contenant des actions publiques et privées.
- Les actions privées ne nécessitent pas de coordination avec les autres agents.
- Des dépendances sont en revanche à prendre en compte entre les actions publiques des agents.
- A partir des interactions entre actions publiques, il est possible de définir un graphe d'interaction :
 - chaque nœud est une action
 - il existe un arc entre $a_i \in \mathcal{A}_I$ et $a_j \in \mathcal{A}_I$ si $eff(a_i) \cup pre(a_j) \neq \emptyset$.

Remarque : ne permet pas de représenter des interactions liées à la simultanéité des actions.

Représentation des interactions liées à la simultanéité

1ère approche : définir le problème en considérant les actions jointes et non plus les actions individuelles

$\text{JointMove}(b,x,y, \text{ag1}, \text{ag2}) \leftrightarrow \text{Move}(b, x, y, \text{ag1}) \text{ Move}(b, x, y, \text{ag2})$

Action ($\text{JointMove}(b,x,y, \text{ag1}, \text{ag2})$,

Pre : $\text{On}(b,x) \wedge \text{Clear}(b) \wedge \text{Clear}(y) \wedge \text{HeavyBlock}(b) \wedge (b \neq x) \wedge (b \neq y) \wedge (x \neq y) \wedge \text{Agent}(\text{ag1}) \wedge \text{Agent}(\text{ag2})$

Effect : $\text{On}(b,y) \wedge \text{Clear}(x) \wedge \neg \text{On}(b,x) \wedge \neg \text{Clear}(y)$)

Problématique : pour x actions individuelles et n agents $:x^n$ actions jointes possibles alors que beaucoup d'actions individuelles peuvent être indépendantes les unes des autres

Représentation des interactions liées à la simultanéité

2ème approche : définir les actions jointes de façon implicite en ajoutant une liste d'actions concurrentes aux actions STRIPS

Action (Move(b,x,y, ag1)

Concurrent: Move(b,x,y, ag2) $\wedge$ Agent(ag2) $\wedge$ (ag1 $\neq$ ag2)

Pre : On(b,x) $\wedge$ Clear(b) $\wedge$ Clear(y) $\wedge$ HeavyBlock(b) $\wedge$ (b $\neq$ x)
 $\wedge$ (b $\neq$ y) $\wedge$ (x $\neq$ y) $\wedge$ Agent(ag1)

Effect : On(b,y) $\wedge$ Clear(x) $\wedge$ $\neg$ On(b,x) $\wedge$ $\neg$ Clear(y))

PDDL multi-agent [Kov12]

- Planning Domain Definition Language (PDDL) : langage de description des problèmes de planification utilisé pour International Planning Competition (IPC).
- Extension multi-agent :
 - Modélisation d'actions concurrentes avec des interactions
 - Modélisation de domaines compétitifs, coopératifs, mixtes.
 - Les agents peuvent avoir des actions/buts/utilités différentes
 - Modélisation de l'observabilité totale et/ou partielle
 - Hérite des caractéristiques de PDDL
- Possibilité de définir un fichier par agent pour permettre la planification distribuée et modéliser les informations privées des agents.

PDDL multi-agent [Kov12]

Définition du domaine de planification dans 1 fichier :

```
(define (domain ma-lift-table)
  (:requirements equality :negative-preconditions
    :existential-preconditions :typing :multi-agent)
  (:types agent) (:constants table)
  (:predicates (lifted ?x - object) (at ?a - agent ?o - object))
  (:action lift :agent ?a - agent :parameters ()
    :precondition (and (not (lifted table)) (at ?a table)
      (exists (?b - agent)
        (and (not (= ?a ?b)) (at ?b table) (lift ?b))))
    :effect (lifted table)))
```

Définition d'une action soulever la table qui nécessite la coopération de agents.

PDDL multi-agent [Kov12]

Définition du problème de planification dans 1 ou plusieurs fichiers :

```
(define (problem ma-lift-table-1)
  (:domain ma-lift-table)
  (:objects a b - agent)
  (:init (at a table) (at b table))
  (:goal :agent agent :condition (lifted table)))
```

Deux agents doivent résoudre la tâche qui consiste à soulever la table.

Le plan qui devrait être calculé est de la forme :

[0:(lift a) 0:(lift b)]

Allocation des buts et tâches aux agents

- Les méthodes de planification centralisée réalise généralement l'allocation des tâches en même temps que la planification.
- Il existe un grand nombre de méthodes d'allocation des tâches distribuées :
 - Contract Net
 - Enchères
 - Simulations de marchés économiques...
- L'objectif des agents est le plus souvent de maximiser leur propre efficacité ou l'efficacité du système (vs. équité entre les agents).

Contract-Net

- Protocole distribué de négociation pour l'allocation de tâches
- Protocole basé sur des appels d'offres
- 2 rôles : le gestionnaire et le contractant
- Le gestionnaire a une tâche à réaliser et souhaite la sous-traiter à un autre agent.
- Chaque agent peut endosser chacun des 2 rôles

Contract-Net

Différentes phases du protocole :

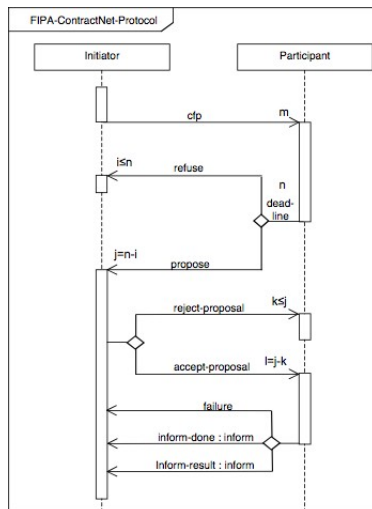
- ① Le gestionnaire envoie un appel d'offre à tous les contractants : description de la tâche à réaliser.
- ② Les contractants répondent par une offre : valeur numérique ou vecteur de valeurs.
- ③ Le gestionnaire sélectionne la meilleure offre reçue.
- ④ Le gestionnaire établit le contrat avec le contractant ayant envoyé la meilleure offre.

Contract Net

Reprenons le protocole Contract Net et étudions les problématiques posées par :

- Distribution : en particulier le fait de devoir communiquer et que la communication en soit pas parfaite et la parallélisation des exécutions des agents
- Asynchronisme : en particulier pour l'exécution des tâches et la communication
- Ouverture : en communication limitée, on peut considérer que les agents entrent et sortent du champ de communication

Contract-Net



Contract-Net

Problématiques liées au Contract-Net

- Comment gérer les communications avec retard ou perte de messages ?
- Comment répondre à un appel d'offre ? Calcul d'un plan optimal ? Heuristique ?
- Est-il nécessaire de connaître tous les contractants potentiels ?
- Comment gérer les appels d'offres simultanées ou successifs ?

Enchères

- Interaction entre un auctioneer (commissaire priseur) et des bidders (enchérisseurs)
- Le but pour le commissaire priseur est d'allouer les tâches aux enchérisseurs.
- Différents protocoles qui varient selon :
 - la méthode de détermination du gagnant : first-price, second-price
 - la diffusion des montants enchéris : enchères publiques ou privées
 - le mécanisme utilisé pour réaliser les enchères : enchères en un tour, enchères ascendantes, enchères descendantes

Enchères

Enchères anglaises

“first-price, open cry, ascending”

- Le commissaire priseur annonce un prix de réserve.
- Les agents sont invités à enchérir plus que l'enchère courante.
- Quand plus aucun enchérisseur ne veut enchérir, la tâche est allouée à celui qui a fait la plus forte offre au prix qu'il a proposé.

Enchères

Enchères hollandaises

“open cry, descending”

- Le commissaire priseur annonce un prix artificiel excessivement haut.
- Le commissaire priseur diminue le prix d'un petit montant tant qu'aucun agent ne fait d'offre sur la tâche.
- Dès qu'un agent fait une offre, la tâche est allouée à cet agent.

Enchères

Enchères de Vickrey

“second-price sealed-bid”

- Un seul tour de négociation au cours duquel chaque enchérisseur fait une seule enchère.
- Les enchérisseurs ne voient pas les offres faites par les autres.
- La tâche est allouée à l'agent ayant fait la plus forte offre mais au prix de la seconde meilleure offre.

Avantage : il est prouvé que la stratégie dominante de chaque agent est de révéler sa vraie valeur pour la tâche → très utilisé dans les systèmes multi-agents

Question : pourquoi révéler sa vraie valeur est une stratégie dominante ?

Enchères

Problématiques liées aux enchères :

- Comment déterminer sa valeur pour une tâche ?
- Complexité de communication entre les agents ?
- Incertitude sur les valeurs des enchères ?
- Comment allouer une tâche à plusieurs agents ? (coalitions)
[SK98]

Allocation des tâches

Autres pistes :

- Tenir compte des capacités des agents
- Tenir compte des dépendances entre les tâches.
- Tenir compte des contraintes spatiales et temporelles sur les tâches et les agents.

Il existe une grande littérature à ce sujet en lien avec le choix social computationnel, l'ordonnancement et la théorie des jeux.

Références I



Ronen I. Brafman and Carmel Domshlak, *From one to many: Planning for loosely coupled multi-agent systems*, Proceedings of the Eighteenth International Conference on International Conference on Automated Planning and Scheduling, ICAPS'08, AAAI Press, 2008, pp. 28–35.



Marie desJardins, Edmund H Durfee, Jr. Ortiz, and Michael J Wolverton, *A survey of research in distributed, continual planning*, AI Magazine **20** (1999), 13 – 13.



J. Ferber, *Les systèmes multi-agents : Vers une intelligence collective*, Inter Editions, 1995.



E.A Hansen, D. Bernstein, and S. Zilberstein, *Dynamic programming for partially observable stochastic games*, Proceedings of the Nineteenth National Conference on Artificial Intelligence, 2004.



Bryan Horling and Victor Lesser, *A survey of multi-agent organizational paradigms*, Knowl. Eng. Rev. **19** (2004), no. 4, 281–316.

Références II



Daniel L Kovacs, *A multi-agent extension of pddl3.1*, Proceedings of the 3rd Workshop on the International Planning Competition (IPC), 22nd International Conference on Automated Planning and Scheduling (ICAPS-2012), ICAPS, 2012, pp. 19–27.



T.W Malone, *What is coordination theory*, National Science Foundation Coordination Theory Workshop, 1988.



S. Russell and P. Norvig, *Artificial intelligence : A modern approach*, Prentice Hall Series, 2003.



N. Sabouret, *Interactions sur le fonctionnement dans les systèmes multi-agents ouverts et hétérogènes*, 2009.



Onn Shehory and Sarit Kraus, *Methods for task allocation via agent coalition formation*, Artificial Intelligence **101** (1998), no. 1, 165 – 200.

Références III



Alejandro Torreño, Eva Onaindia, Antonín Komenda, and Michal Stolba, *Cooperative multi-agent planning: A survey*, CoRR **abs/1711.09057** (2017).



J. Von Neumann and O. Morgenstern, *Theory of games and economic behavior*, Princeton University Press, 1944.



G Weiss, *Multiagent systems a modern approach to distributed artificial intelligence*, MIT Press, 1999.



M. Wooldridge, *An introduction to multiagent systems*, John Wiley and Sons, 2002.